Slide 1: Opening

Main goal

Turn a proposed model change into an experiment that supports an accept, revise or reject decision. **Opening words**

We have discussed training methods. Now we will decide what needs to change, inspect the training target, protect the comparison and examine what actually improved or deteriorated. A completed training run produces a candidate; it does not qualify that candidate for use. **Simple example**

A request was waiting yesterday and approved today. The system keeps reporting the old status. We need to find out whether the missing update, the interpretation or a later application handoff caused the problem. **Participation**

The recorded adaptation notebook and the controlled repair exercise require no API key or GPU. Optional instructor training is a separate, deliberately enabled activity. The additional pages unfold each concept progressively; they do not require extra live model calls.

Slide 2: Start with the behaviour you want to change

Main goal in plain English

Name a checkable failure.

Natural opening and explanation

“Make the model better” does not tell us what to train or how to judge success. Choose a recurring behaviour that a reviewer can recognize.

We need a precise behaviour before choosing a training method. Ask what observation would convince us that the problem has been fixed.

Concrete example

A request was waiting yesterday and approved today. The model keeps reporting the old status.

Detailed explanation for delivery

“Instead of saying make the assistant smarter, we can say: when the record contains an old status and a later update, return the later supported status. Now a reviewer can examine each answer and tell us whether the intended behaviour occurred.”

“A useful adaptation experiment starts with a behaviour we want to change. ‘Make the model better’ is too vague. ‘Select the current status rather than the superseded status’ can be checked.”

Read the opening question and pause after “repeatable behaviour.” One attractive answer is not enough: the failure should recur across examples representative of the intended use. Then explain “what would count as evidence.” We need an input contract, reviewed targets, a baseline and a method of measuring both improvements and new errors.

Caveat or limitation

Avoid treating training as the default solution. A missing source, corrupted calculator input or permission defect may be repairable without modifying the model.

Transition

Next we will consider this point: A training run produces a candidate.

Slide 3: A training run produces a candidate

Main goal in plain English

Decide whether to use it.

Natural opening and explanation

We compare the candidate with the unchanged model on the same cases. Count both errors it fixes and errors it introduces.

Accept, revise and reject are all legitimate experiment outcomes. Finishing training does not decide among them.

Concrete example

Correctly reading the new approval is useful only if the change does not also invent approvals when no update exists.

Detailed explanation for delivery

"A useful adaptation experiment starts with a behaviour we want to change. 'Make the model better' is too vague. 'Select the current status rather than the superseded status' can be checked."

Read the opening question and pause after "repeatable behaviour." One attractive answer is not enough: the failure should recur across examples representative of the intended use. Then explain "what would count as evidence." We need an input contract, reviewed targets, a baseline and a method of measuring both improvements and new errors.

For the first bullet, distinguish the session's purpose from Section 3. Section 3 described training mechanisms. Here we decide what to train, how to compare it and whether to accept the result. For the second bullet, explain that the training job produces a candidate. The candidate still has to face the same cases as the baseline. The result may be accept, revise or reject; all three are useful outcomes when supported by evidence.

Caveat or limitation

Avoid treating training as the default solution. A missing source, corrupted calculator input or permission defect may be repairable without modifying the model.

Transition

Next we will consider this point: Our route: six decisions in order.

Slide 4: Our route: six decisions in order

Main goal in plain English

Our route: six decisions in order

Natural opening and explanation

The route keeps every technical step connected to a decision. We will use saved training artifacts and a controlled repair exercise; fresh GPU training is optional.

Detailed explanation for delivery

"For the request-status problem, we first confirm that the update reached the model. Then we write the desired answer, reserve unseen records, choose an update method, compare answers and decide whether the result is good enough. Skipping the first step could make us train around a missing-document problem."

Suggested pacing for the hour: about ten minutes for diagnosis and specification, ten for data and loss, ten for configuration, fifteen for recorded outputs, ten for workflow repair and five for the decision and questions. Adapt that pacing to the audience; preserve the hands-on inspection time.

This is a practical sequence, not a claim that development never iterates. Iteration is expected; record which evidence was used at each decision and obtain fresh assessment evidence when necessary.

Caveat or limitation

This is a practical sequence, not a claim that development never iterates. Iteration is expected; record which evidence was used at each decision and obtain fresh assessment evidence when necessary.

Transition

Next we will consider this point: What you will be able to explain.

Slide 5: What you will be able to explain

Main goal in plain English

What you will be able to explain

Natural opening and explanation

These are practical abilities. We will check checkpoint selection and repair choice with two questions, and inspect actual outputs in the exercises.

Detailed explanation for delivery

"At the end, you should be able to point to a request record, identify the desired response, explain what part of the model was allowed to change, compare two outputs and say whether the change fixed the problem or created another one."

First, a measurable adaptation hypothesis should state an observed behaviour, the proposed change and the result that would support it. Second, explain two different selections: which token positions contribute to the loss, and which model parameters are permitted to update. Third, protect the distinction between fitting parameters, selecting a checkpoint and assessing the selected model. Fourth, compare the same cases before and after adaptation, counting errors fixed and new errors introduced. Fifth, trace a workflow failure to a specific observable handoff and select a plausible repair.

Tell participants what they will produce. In the first exercise they will identify a changed prediction and decide whether it is a correction or a regression. In the second they will inspect a faulty tool request, repair one value and explain which downstream outputs changed. The multiple-choice questions test checkpoint selection and choosing the right repair.

Caveat or limitation

Knowing how to call a trainer is not sufficient evidence that these objectives were met. Ask participants to point to the relevant artifact or output.

Transition

Next we will consider this point: First check whether the information arrived.

Slide 6: First check whether the information arrived

Main goal in plain English

Missing evidence calls for retrieval.

Natural opening and explanation

If the latest update never reached the model, there is no supplied update for it to interpret. Inspect the actual context before changing weights.

Retrieve the missing update and repeat the same task. If the error disappears, that is evidence about this workflow repair, not a successful model adaptation.

Concrete example

The document collection contains today's approval, but the prompt includes only yesterday's waiting status.

Detailed explanation for delivery

"Suppose the record says approved and the model correctly extracts approved, but a later program writes waiting into the database. More examples of approved requests do not directly repair that program. We should correct the handoff and check that the database receives the extracted value."

Read each row as a diagnostic branch. If the required information is absent from the context, obtain it or repair retrieval. Training cannot be assumed to recover a fact that was never supplied. If the instruction is unclear, test a clarified task and compare outputs under otherwise identical conditions. If adequate context and a clear instruction still lead to repeated interpretation errors, curated demonstrations and adaptation become plausible.

Use a simple example. A document says a parcel is now approved; the model reads an earlier "waiting" status. That might be an interpretation problem. If the model returns "approved" but application code sends "waiting" to the next step, the error lies in the handoff. The same final status can conceal different causes.

Caveat or limitation

These are candidates for controlled tests, not automatic prescriptions. Record the original input and output before changing anything. Several defects can coexist.

Transition

Next we will consider this point: Then locate the remaining failure.

Slide 7: Then locate the remaining failure

Main goal in plain English

Interpretation and handoff errors need different tests.

Natural opening and explanation

Repeatedly selecting the old status from correct context can justify adaptation. Correct extraction followed by a corrupted tool argument calls for application repair.

Also check instructions and authority. An unclear instruction can be clarified; a correct proposal still needs permission to execute.

Concrete example

Model output: approved. Database request: waiting. Repair the code that builds the request.

Detailed explanation for delivery

Read each row as a diagnostic branch. If the required information is absent from the context, obtain it or repair retrieval. Training cannot be assumed to recover a fact that was never supplied. If the instruction is unclear, test a clarified task and compare outputs under otherwise identical conditions. If adequate context and a clear instruction still lead to repeated interpretation errors, curated demonstrations and adaptation become plausible.

The fourth row is a different kind of defect: the model extracts the right figure, but application code changes or misroutes it when constructing a tool call. Inspect the handoff and validate the arguments. The final row concerns authority: a correct proposal can still be unauthorized, so execution permissions must be enforced by the controller.

Use a simple example. A document says a parcel is now approved; the model reads an earlier "waiting" status. That might be an interpretation problem. If the model returns "approved" but application code sends "waiting" to the next step, the error lies in the handoff. The same final status can conceal different causes.

Caveat or limitation

These are candidates for controlled tests, not automatic prescriptions. Record the original input and output before changing anything. Several defects can coexist.

Transition

Next we will consider this point: Write the input and target before training.

Slide 8: Write the input and target before training

Main goal in plain English

Make the contract explicit.

Natural opening and explanation

Define what information the model receives, what response is acceptable and how missing or contradictory information should be represented.

Concrete example

Input: old status + latest update. Target: current status + supporting words. If no current update exists, return unknown.

Detailed explanation for delivery

"Write this table before running the experiment. It protects us from changing the definition of success after seeing the results."

The primary metric is correct current-status extraction on records kept outside training and selection. Define how partial answers and missing evidence will be scored before evaluation. Regression checks examine cases where a favourable aggregate can hide deterioration: missing status, contradictory records or unfamiliar phrasing.

For example, "Earlier: pending. Latest update: approved" should yield approved. A record containing only "Earlier: pending" should not automatically be labelled approved. If the task requires an evidence quote, both the status and the quote should be inspected. Explain that a target format can include an explicit unknown state when the required information is absent.

Caveat or limitation

This status example is an illustrative design, not a reported experiment. Later, the real saved workshop run concerns a different classification task. Do not transfer its numerical results to this example.

Transition

Next we will consider this point: Define success before inspecting outcomes.

Slide 9: Define success before inspecting outcomes

Main goal in plain English

Keep the baseline and assessment fixed.

Natural opening and explanation

Use the unchanged model with the same prompt and evidence. Evaluate unseen records and inspect difficult subsets, including missing updates and contradictory records.

Choose metrics before the run. Otherwise it is easy to select whichever metric happens to improve.

Concrete example

A candidate that improves easy approvals but invents approval on missing-update cases may fail the intended task.

Detailed explanation for delivery

"We can prepare twenty new request records, some with clear updates and some with no update. We decide in advance how to score approved, waiting and unknown. We then give the unchanged and adapted models the same records. The number twenty is illustrative, not the size of a reported experiment."

Explain the input contract: the model will receive a short record containing an old status and a current update. The target is to return the current status and quote its supporting words. This is specific enough for a reviewer to check. The baseline is the unchanged model using the same task prompt and context. That comparison helps isolate the model change.

The primary metric is correct current-status extraction on records kept outside training and selection. Define how partial answers and missing evidence will be scored before evaluation. Regression checks examine cases where a favourable aggregate can hide deterioration: missing status, contradictory records or unfamiliar phrasing.

Caveat or limitation

This status example is an illustrative design, not a reported experiment. Later, the real saved workshop run concerns a different classification task. Do not transfer its numerical results to this example.

Transition

Next we will consider this point: One labelled example: what is input, what is target?.

Slide 10: One labelled example: what is input, what is target?

Main goal in plain English

One labelled example: what is input, what is target?

Natural opening and explanation

During training we supply all three messages and score the target response. At inference we supply the instruction and input, then ask the model to produce the answer. This is an illustrative record, not a claimed executed training run.

Detailed explanation for delivery

Read the input aloud: "Old status: waiting. Update: the request was approved." Then read the instruction: extract the current status and quote the supporting words. The target response contains two requirements. "Status: approved" teaches the desired decision, and the quoted evidence teaches the desired support format.

Here is a readable conversational record. The roles identify who supplied each piece:

> System: Extract the latest supported status. If there is no current update, return unknown.
> User: Old status: waiting. Latest update: the request was approved.
> Assistant target: Status: approved. Evidence: the request was approved.

"The user message is an input. The assistant message is the reviewed answer we want the model to imitate. During inference, we supply the first two messages and ask the model to produce the third. During training, we also provide the third so its tokens can be scored. We are teaching a response pattern and an evidence-selection behaviour. We are not supplying a certificate that every approval claim in future inputs is true."

A second illustrative labelled pair is: input "Old status: waiting. No current update supplied"; target "Status: unknown. Evidence: no current update supplied." Its purpose is to teach the defined missing-evidence response. It is not a live training result. The dataset must contain a sufficient range of situations for the intended claim; two examples alone do not establish generalization.

Caveat or limitation

The reviewed target is not a guarantee of truth. Its reliability depends on how it was produced and checked. The model learns token prediction from the supplied demonstrations.

Transition

Next we will consider this point: The target teaches the behaviour we reward.

Slide 11: The target teaches the behaviour we reward

Main goal in plain English

A wrong target teaches the wrong answer.

Natural opening and explanation

Training increases the probability of reviewed response tokens. Including a supporting quotation teaches a different output contract from training on a status word alone.

The target needs review and a consistent definition. It is not a guarantee that every future output is true.

Concrete example

Input: old status waiting; no current update. ewline Reviewed target: Status: unknown. Evidence: no current update supplied.

Detailed explanation for delivery

"The input says old status waiting, latest update approved. Our reviewed answer says current status approved and quotes the update. If we accidentally label that record waiting, training rewards the wrong answer. Reviewing targets is therefore part of the method, not administrative cleanup."

Explain why the target matters. Training only on "approved" teaches a different output contract from training on the status plus a quote. A confident but incorrect target is still a target the optimizer will learn to imitate. Therefore the records need review, consistent definitions and appropriate handling of ambiguity.

Here is a readable conversational record. The roles identify who supplied each piece:

> System: Extract the latest supported status. If there is no current update, return unknown.
> User: Old status: waiting. Latest update: the request was approved.
> Assistant target: Status: approved. Evidence: the request was approved.

"The user message is an input. The assistant message is the reviewed answer we want the model to imitate. During inference, we supply the first two messages and ask the model to produce the third. During training, we also provide the third so its tokens can be scored. We are teaching a response pattern and an evidence-selection behaviour. We are not supplying a certificate that every approval claim in future inputs is true."

Caveat or limitation

The reviewed target is not a guarantee of truth. Its reliability depends on how it was produced and checked. The model learns token prediction from the supplied demonstrations.

Transition

Next we will consider this point: How a reviewed response changes the model.

Slide 12: How a reviewed response changes the model

Main goal in plain English

How a reviewed response changes the model

Natural opening and explanation

Follow the prediction, loss and gradient arrows. The objective rewards matching the supplied target, not directly truth or financial profit.

Detailed explanation for delivery

The prompt asks for an explanation of deep learning. The partial reviewed response ends with "neural." The pretrained model processes that context and predicts the next token. The target shown is "networks." The loss is larger when the model assigns that target low probability. The red arrow labelled gradients represents information used to change the trainable parameters in a direction intended to reduce the loss.

The displayed formula has a negative sum of log probabilities: high probability assigned to the reviewed tokens produces a smaller loss. "Distance" in the figure's explanatory concept should not be interpreted as a direct measure of factual correctness. It is a prediction objective.

"We model the answer as an ordered sequence of tokens. At each position the model produces a probability distribution over its vocabulary. We treat the reviewed next token as the target. The loss therefore rewards matching that target, conditional on the preceding context. It does not directly reward factual truth, profit or a valid permission decision." The gradient is a local sensitivity of the differentiable loss to a parameter. An optimizer uses it to propose a small change; that does not guarantee a globally optimal solution or an improvement on unseen examples. When using LoRA, only the selected trainable parameters are updated. This is an assumption about the permitted update, not evidence that the chosen components caused the original error.

Caveat or limitation

The figure does not imply that every parameter must update. With LoRA, gradients are used to update selected adapter parameters while the original weights remain frozen.

Transition

Next we will consider this point: The loss scores the response tokens we choose.

Slide 13: The loss scores the response tokens we choose

Main goal in plain English

The loss scores the response tokens we choose

Natural opening and explanation

A token can be a word, part of a word or punctuation. θ denotes model parameters, $y_{<t}$ earlier target tokens, and the denominator counts supervised positions. The mask must select at least one position.

Detailed explanation for delivery

Define every symbol. θ denotes the model parameters being optimized. x is the input: instruction and context. y_t is the target response token at position t. $y_{<t}$ means all target response tokens before that position. $p_\theta(y_t \mid x, y_{<t})$ is the probability the model gives the reviewed next token in that context. The logarithm converts a probability into an additive score; the minus sign turns higher probability into a smaller penalty.

The mask m_t is one for a supervised position and zero for a position to ignore. Multiplying by zero removes that position from the sum. Dividing by $\sum_t m_t$ averages across the supervised positions. That denominator must be nonzero: an example with no supervised tokens is a data-processing problem.

Teacher forcing means the earlier response tokens are the reviewed target prefix during loss calculation. It gives a tractable training objective, but the model faces its own generated prefixes at deployment. A model may score a target well when supplied the right prefix and still make an early generation error. An end-of-sequence token may also be part of the supervised target; its inclusion must be checked.

> Input tokens: [system instruction] [user record] — loss mask 0.
> Target tokens: [Status] [:] [approved] [end] — loss mask 1.

These bracketed units illustrate positions; they are not an assertion about a specific tokenizer's actual token IDs. The input remains visible as context despite receiving zero direct loss. The denominator in the displayed loss is the number of supervised positions. A dataset objective also needs a weighting rule. Averaging all supervised tokens gives longer responses more influence; averaging each example's mean loss gives examples equal influence. Repeating or oversampling a record increases its influence. These objectives need not select the same candidate.

Caveat or limitation

Exact aggregation and normalization can vary by trainer and batching configuration. Inspect the actual implementation. The expression here describes the intended supervised-token average, not a universal promise about every library default.

Transition

Next we will consider this point: Teacher forcing supplies the correct earlier words.

Slide 14: Teacher forcing supplies the correct earlier words

Main goal in plain English

Teacher forcing supplies the correct earlier words

Natural opening and explanation

The product is the chain rule for an ordered sequence, not an independence assumption. Autoregressive prediction uses earlier context, not later target tokens. Good target-token loss need not guarantee a correct free-generated answer.

Detailed explanation for delivery

Explain teacher forcing: when scoring the token "approved," earlier reviewed response tokens are provided, even if the model would not have generated them correctly on its own. Free generation instead feeds back its own selected tokens. That is why good training loss does not by itself establish a reliable complete answer.

The probability of a complete target response can be written

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}).$$

"T is the number of response tokens. The product combines the probabilities of all successive target tokens. This factorization follows the probability chain rule; it does not assume the tokens are independent. Our autoregressive implementation assumes that the prediction at a position uses the input and earlier tokens, with no access to later target tokens. A finite model context can restrict which earlier information is available."

Teacher forcing means the earlier response tokens are the reviewed target prefix during loss calculation. It gives a tractable training objective, but the model faces its own generated prefixes at deployment. A model may score a target well when supplied the right prefix and still make an early generation error. An end-of-sequence token may also be part of the supervised target; its inclusion must be checked.

> Input tokens: [system instruction] [user record] — loss mask 0.
> Target tokens: [Status] [:] [approved] [end] — loss mask 1.

These bracketed units illustrate positions; they are not an assertion about a specific tokenizer's actual token IDs. The input remains visible as context despite receiving zero direct loss. The denominator in the displayed loss is the number of supervised positions. A dataset objective also needs a weighting rule. Averaging all supervised tokens gives longer responses more influence; averaging each example's mean loss gives examples equal influence. Repeating or oversampling a record increases its influence. These objectives need not select the same candidate.

Caveat or limitation

Exact aggregation and normalization can vary by trainer and batching configuration. Inspect the actual implementation. The expression here describes the intended supervised-token average, not a universal promise about every library default.

Transition

Next we will consider this point: The loss mask does not hide the input.

Slide 15: The loss mask does not hide the input

Main goal in plain English

The loss mask does not hide the input

Natural opening and explanation

Inspect the rendered sequence and supervised-token mask. Ignoring input loss does not remove input context.

Detailed explanation for delivery

"Suppose the input contains twenty words and the response is status approved. We may want the two response words to contribute to the loss while the input words provide context only. Ignoring an input position in the loss does not remove that word from the model's context."

Teacher forcing means the earlier response tokens are the reviewed target prefix during loss calculation. It gives a tractable training objective, but the model faces its own generated prefixes at deployment. A model may score a target well when supplied the right prefix and still make an early generation error. An end-of-sequence token may also be part of the supervised target; its inclusion must be checked.

> Input tokens: [system instruction] [user record] — loss mask 0.
> Target tokens: [Status] [:] [approved] [end] — loss mask 1.

These bracketed units illustrate positions; they are not an assertion about a specific tokenizer's actual token IDs. The input remains visible as context despite receiving zero direct loss. The denominator in the displayed loss is the number of supervised positions. A dataset objective also needs a weighting rule. Averaging all supervised tokens gives longer responses more influence; averaging each example's mean loss gives examples equal influence. Repeating or oversampling a record increases its influence. These objectives need not select the same candidate.

Caveat or limitation

Exact aggregation and normalization can vary by trainer and batching configuration. Inspect the actual implementation. The expression here describes the intended supervised-token average, not a universal promise about every library default.

Transition

Next we will consider this point: How examples are weighted changes what is learned.

Slide 16: How examples are weighted changes what is learned

Main goal in plain English

Specify the averaging rule.

Natural opening and explanation

Averaging all supervised tokens gives long responses more influence. Averaging each example's mean gives examples equal influence. Repeating a record increases its weight.

The dataset objective and sampling scheme determine influence. Lower loss means better fit to that chosen weighting, not automatically calibrated real-world outcome probabilities.

Concrete example

Oversampling a rare class can balance training, but it does not create new independent information.

Detailed explanation for delivery

Teacher forcing means the earlier response tokens are the reviewed target prefix during loss calculation. It gives a tractable training objective, but the model faces its own generated prefixes at deployment. A model may score a target well when supplied the right prefix and still make an early generation error. An end-of-sequence token may also be part of the supervised target; its inclusion must be checked.

> Input tokens: [system instruction] [user record] — loss mask 0.
> Target tokens: [Status] [:] [approved] [end] — loss mask 1.

These bracketed units illustrate positions; they are not an assertion about a specific tokenizer's actual token IDs. The input remains visible as context despite receiving zero direct loss. The denominator in the displayed loss is the number of supervised positions. A dataset objective also needs a weighting rule. Averaging all supervised tokens gives longer responses more influence; averaging each example's mean loss gives examples equal influence. Repeating or oversampling a record increases its influence. These objectives need not select the same candidate.

"We assume the reviewed labels are appropriate examples of the intended task and that the weighting represents what we want to learn. If labels are noisy or oversampling changes class balance, a lower loss still means better fit to that chosen objective, not calibrated real-world probabilities." Inspect the trainer's actual reduction and sampling configuration rather than assuming the slide fixes those implementation choices.

Caveat or limitation

Exact aggregation and normalization can vary by trainer and batching configuration. Inspect the actual implementation. The expression here describes the intended supervised-token average, not a universal promise about every library default.

Transition

Next we will consider this point: Inspect the processed training record.

Slide 17: Inspect the processed training record

Main goal in plain English

The model sees tokens, not your original file.

Natural opening and explanation

Check role separators, the response positions selected for loss and the sequence after truncation. A correct source record can become an incomplete model input.

Check padding too: padded positions should not become desired response language. Use compatible training and inference templates.

Concrete example

A long request history ends with “approved.” A length limit removes that final update. The remaining input no longer supports approval.

Detailed explanation for delivery

“A long request history ends with the crucial sentence the request was approved. If the length limit cuts off that sentence, the model cannot use it. A correct original document can therefore become an incomplete training input. We inspect the processed example to catch that.”

Walk down the four checks. Roles and separators identify where the system instruction, user message and assistant response begin and end. The training and inference templates should be compatible. Visible target tokens let us confirm that the intended response is actually present and supervised.

Maximum sequence length is a practical limit: truncation may remove the decisive exception or the entire target. A long document with its most important qualification at the end is an easy example. Padding fills unused positions so examples can be batched; padding should not become a desired language target. The loss mask determines which positions actually affect the objective.

Caveat or limitation

Do not equate a successful tokenizer call with a valid training example. The sequence can be syntactically valid while the target is missing or the wrong positions are supervised.

Transition

Next we will consider this point: Give each dataset one job.

Slide 18: Give each dataset one job

Main goal in plain English

Fit, select, then assess.

Natural opening and explanation

Training adjusts parameters. Validation chooses settings or a checkpoint. The final test assesses the selected candidate after those choices are frozen.

Separation concerns information use, not file names. Repeatedly choosing settings from final-test performance turns the final test into development evidence.

Concrete example

Practice questions teach; a mock exam guides selection; a final exam checks the chosen preparation.

Detailed explanation for delivery

"Think of practice questions, a mock exam and a final exam. Practice adjusts the learner. The mock exam helps choose what to practise next. If we repeatedly choose our approach using final-exam answers, that exam no longer gives an independent check. Related copies of the same question also weaken the separation."

Training records are used to adjust parameters. Validation records help choose settings or a checkpoint. Final-test records assess the selected procedure after those choices are frozen. Explain that the separation is about information use, not merely different filenames.

Near-duplicates and related documents can carry the same information across partitions. For example, two versions of one report may differ in punctuation but share almost all facts. If one is in training and the other is in the final test, success may reflect familiarity rather than generalization to a genuinely new record. Grouping related items can make the assessment better match the intended deployment.

Caveat or limitation

A chronological split alone does not establish absence of leakage. Revised sources and pretrained-model knowledge can complicate historical assessment. Avoid making a stronger claim than the split supports.

Transition

Next we will consider this point: Separate rows may still share information.

Slide 19: Separate rows may still share information

Main goal in plain English

Match the split to the claim.

Natural opening and explanation

Copies, revised documents, repeated entities and overlapping outcome windows can cross a row split. A later period with familiar entities does not test new-entity generalization.

For future-outcome labels, the outcome must have matured before training. The saved experiment uses the same four issuers and overlapping windows; a date split does not make them independent.

Concrete example

Training on yesterday's version of a record and testing on a near-identical revision may reward familiarity.

Detailed explanation for delivery

Near-duplicates and related documents can carry the same information across partitions. For example, two versions of one report may differ in punctuation but share almost all facts. If one is in training and the other is in the final test, success may reflect familiarity rather than generalization to a genuinely new record. Grouping related items can make the assessment better match the intended deployment.

"The comparison is useful only if the held-out records represent the situations we intend to handle. Independent rows are a convenient statistical assumption, but rows can share a document, entity, time window or outcome. Treating many related rows as independent exaggerates the amount of evidence." Grouping by document or entity, and separating periods where appropriate, makes the assessment match a more explicit generalization claim. A temporal split with the same entities tests later observations for those entities; it does not test entirely new entities. Distribution shifts can still make future performance differ.

For a future-outcome target, training requires both an eligible historical input and a target that had matured by training time. Later revisions to a source must not silently replace what was available. The saved workshop comparison is retrospective, uses the same four issuers across periods and has overlapping training windows even after identical-window deduplication. Those are limitations of the evidence, not assumptions we can declare true by choosing a split date.

Caveat or limitation

A chronological split alone does not establish absence of leakage. Revised sources and pretrained-model knowledge can complicate historical assessment. Avoid making a stronger claim than the split supports.

Transition

Next we will consider this point: Question: which result should select the checkpoint?.

Slide 20: Question: which result should select the checkpoint?

Main goal in plain English

Question: which result should select the checkpoint?

Natural opening and explanation

A checkpoint is a saved model state. Choose an answer before moving to the next page. Ask whether the evidence used for selection remains an independent final assessment.

Detailed explanation for delivery

"Suppose we save a candidate after each of three passes through the training records. We need a rule for choosing one. If we inspect the final-test answers each time and select the best, we have used those answers to develop the model. That is the issue the question is testing."

If someone chooses A, ask whether the final-test result has now influenced which model is selected. That question directs attention to information flow without immediately revealing the answer. If someone chooses C, ask what happens to examples on which the candidate deteriorated.

The question assumes a final test intended to remain untouched during development. In a development-only experiment, another dataset can be used for selection, but it must be labelled honestly.

Caveat or limitation

The question assumes a final test intended to remain untouched during development. In a development-only experiment, another dataset can be used for selection, but it must be labelled honestly.

Transition

Next we will consider this point: Answer: B, use validation for selection.

Slide 21: Answer: B, use validation for selection

Main goal in plain English

Keep final assessment separate.

Natural opening and explanation

Validation supports choosing a candidate. Training loss diagnoses fit. The untouched final test assesses that chosen procedure, including its selection decisions.

If final results motivate another change, they have become development feedback. A new final claim requires fresh assessment evidence.

Concrete example

Choose epoch two using validation, freeze it, then assess it on the final records.

Detailed explanation for delivery

"We might choose the second saved candidate because it handles the mock-exam records best. We then freeze that choice and use the final records once. If the final result disappoints, we may learn from it, but a new development cycle needs a new independent assessment."

Training loss tells us whether the optimization process is fitting the supervised examples. It can diagnose a stalled run or a masking error, but it does not establish generalization. Validation provides evidence for choosing a checkpoint or setting. That rule should be declared so selection is systematic rather than based on whichever graph looks attractive.

The final test then assesses the selected procedure, including the choices made using training and validation. If we repeatedly change settings after seeing final-test results, those results have become development feedback. We can still learn from them, but a fresh assessment is needed for a new final claim.

Caveat or limitation

Selection using validation reduces one source of optimistic evaluation; it does not eliminate every bias or guarantee deployment performance.

Transition

Next we will consider this point: LoRA learns a correction to a frozen transformation.

Slide 22: LoRA learns a correction to a frozen transformation

Main goal in plain English

LoRA learns a correction to a frozen transformation

Natural opening and explanation

A vector is a list of numbers. A matrix transforms one list into another. x is the input and h the output; r is rank and α/r scales the correction. SFT supplies the objective; LoRA supplies the update parameterization.

Detailed explanation for delivery

A vector is a list of numbers; a matrix is a rectangular table of numbers that transforms one vector into another.

Read the equation in two parts. W_0x is the original linear transformation: x is the input vector, W_0 is the pretrained weight matrix, and h is the output vector. The added term is $(\alpha/r)BAx$. A first maps the input into a rank-r intermediate space. B maps that intermediate vector into the output space. α/r scales the added contribution.

The expression assumes an ordinary linear transformation with an additive update and standard α/r scaling. Some variants use different scaling, dropout or parameterization. If adapter dropout is active during training, the displayed deterministic expression omits that training-time randomness. A frozen base remains computationally active, and changing an earlier adapter can change activations arriving at later frozen layers. A compatible additive update can be merged for inference, but quantized implementations may require additional care. None of these choices guarantees factual correctness or action-order robustness.

Caveat or limitation

The equation uses the standard scaling convention; implementations and variants can differ. SFT is the objective, LoRA is the update parameterization, and the optimizer is the procedure that adjusts the parameters.

Transition

Next we will consider this point: The same input follows two paths.

Slide 23: The same input follows two paths

Main goal in plain English

The same input follows two paths

Natural opening and explanation

Follow x through frozen W0 and through trainable A then B. The adapter still needs the base model; it is not a smaller independent language model.

Detailed explanation for delivery

Start at the left input box. The upper path uses the frozen pretrained matrix W_0. The lower path passes through trainable A, then trainable B, and receives the scale factor α/r. Both paths meet at the addition box to produce h. The network continues with that combined vector.

Work through the numerical example. One 1024×1024 original matrix contains 1,048,576 entries. With rank eight, A has $8 \times 1024 = 8{,}192$ entries and B has another 8,192. Together that is 16,384 trainable adapter entries. The ratio to this original matrix is 1.5625%. This is an arithmetic illustration for one transformation, not the measured whole-model fraction in the workshop run.

Explain why "between layers" is an imprecise description. The update is attached to chosen linear transformations within the model. In compatible settings, its matrix update can be merged with the original weights for inference. Keeping a separate adapter checkpoint makes it easy to store or switch candidate updates, but the compatible base is still required.

Caveat or limitation

Freezing weights means their stored values are not updated; it does not mean the corresponding computation can be skipped. Gradients still need to reach trainable components through the network.

Transition

Next we will consider this point: Count the parameters that can change.

Slide 24: Count the parameters that can change

Main goal in plain English

Count the parameters that can change

Natural opening and explanation

The count assumes two dense adapter matrices and no extra trainable bias. This is one transformation, not the whole workshop model. Whole-model percentages require counting every trainable tensor and stating the denominator.

Detailed explanation for delivery

"For the illustrated 1024-by-1024 transformation, the original path has just over a million entries. Two rank-eight matrices add 16,384 entries. We can update those added entries while retaining the original transformation. The whole network may contain many other transformations, so its overall percentage must be counted separately."

Work through the numerical example. One 1024×1024 original matrix contains 1,048,576 entries. With rank eight, A has $8 \times 1024 = 8{,}192$ entries and B has another 8,192. Together that is 16,384 trainable adapter entries. The ratio to this original matrix is 1.5625%. This is an arithmetic illustration for one transformation, not the measured whole-model fraction in the workshop run.

The count $r(d_{in} + d_{out})$ assumes two dense adapter matrices, with no additional trainable bias or other trainable modules counted. For the example, $r = 8$ and both dimensions equal 1024. The result is the count for one selected transformation. To obtain a whole-model percentage, sum all actual trainable tensor entries and state the denominator, such as the original base parameter count or combined base-plus-adapter count. These denominators produce slightly different percentages. The savings concern trainable parameters; activation and optimizer memory also depend on sequence length, precision and implementation.

Caveat or limitation

Freezing weights means their stored values are not updated; it does not mean the corresponding computation can be skipped. Gradients still need to reach trainable components through the network.

Transition

Next we will consider this point: LoRA makes an assumption we must test.

Slide 25: LoRA makes an assumption we must test

Main goal in plain English

A restricted update may be enough.

Natural opening and explanation

The product BA has rank at most r. We assume a useful task change can be approximated within that restricted family and in the selected transformations.

The base remains frozen but computationally active. Changing an earlier adapter changes inputs to later layers. Low rank is an efficiency hypothesis, not proof of correctness or a universal optimal choice.

Concrete example

A compact correction may fix current-status extraction. It might also lack the capacity or placement needed for the task.

Detailed explanation for delivery

During ordinary LoRA training the original W_0 is frozen and back-propagation updates A and B. The adapter operates inside selected linear transformations. It is not a new smaller language model placed on top of the original one. Stored base plus adapter contains additional parameters, even though only a small subset is trained.

"We assume that a useful task-specific change can be approximated by a low-rank update in the selected transformations. The product BA has rank at most r: its possible changes are restricted to a smaller family than an unconstrained full matrix update. This is an efficiency hypothesis, not a theorem that the best update for our task has low rank." Freezing W_0 means the optimization cannot directly repair arbitrary features in the base. It can only influence behaviour through the selected adapter paths. We must test whether that restricted family is adequate for the desired behaviour.

The expression assumes an ordinary linear transformation with an additive update and standard α/r scaling. Some variants use different scaling, dropout or parameterization. If adapter dropout is active during training, the displayed deterministic expression omits that training-time randomness. A frozen base remains computationally active, and changing an earlier adapter can change activations arriving at later frozen layers. A compatible additive update can be merged for inference, but quantized implementations may require additional care. None of these choices guarantees factual correctness or action-order robustness.

The equation uses the standard scaling convention; implementations and variants can differ. SFT is the objective, LoRA is the update parameterization, and the optimizer is the procedure that adjusts the parameters.

Caveat or limitation

The equation uses the standard scaling convention; implementations and variants can differ. SFT is the objective, LoRA is the update parameterization, and the optimizer is the procedure that adjusts the parameters.

Transition

Next we will consider this point: Choose which transformations may change.

Slide 26: Choose which transformations may change

Main goal in plain English

Treat target modules as an experiment.

Natural opening and explanation

Query/value projections are one possible configuration. Adding key or other transformations changes the family of permitted updates and often the parameter budget.

Inspect actual module names and trainable tensor counts. Model architectures differ; a configuration string alone is not sufficient evidence.

Concrete example

Compare query/value against query/key/value using the same task and assessment, with a declared compute or parameter budget.

Detailed explanation for delivery

"If our experiment changes only query and value transformations, the key transformations remain fixed. A second experiment could permit keys to change too. To compare them fairly, we keep the task and assessment fixed and declare how we account for the different parameter or compute budget."

Target modules identify which named linear transformations may receive an update. Query and value projections are one possible choice; other choices include keys, outputs or feed-forward transformations. Module names depend on model architecture. Rank controls the intermediate dimension of the update; alpha contributes to its scale. Neither a larger rank nor a larger scale automatically means better task performance.

Ask to inspect the actual list and count of trainable parameters. A configuration can look correct but match unexpected modules. For a scientific comparison of query/value versus another target set, define a budget and use the same task, split, prompts and evaluation. Change one experimental factor at a time where feasible.

Caveat or limitation

The lecture does not establish that query/value updates are optimal for alpha, risk or extraction. Choosing them in a run is a design choice that must be tested.

Transition

Next we will consider this point: Choose the size and duration of the update.

Slide 27: Choose the size and duration of the update

Main goal in plain English

Settings control different things.

Natural opening and explanation

Rank controls update dimension; alpha its scale. Learning rate and steps control optimization. Sequence length and microbatch affect context and memory. Validation schedule controls selection evidence.

Record the seed and environment. More capacity or more training steps do not guarantee improved held-out behaviour.

Concrete example

A longer sequence may retain the decisive exception, while a larger rank does not restore an exception that preprocessing removed.

Detailed explanation for delivery

"If our experiment changes only query and value transformations, the key transformations remain fixed. A second experiment could permit keys to change too. To compare them fairly, we keep the task and assessment fixed and declare how we account for the different parameter or compute budget."

Target modules identify which named linear transformations may receive an update. Query and value projections are one possible choice; other choices include keys, outputs or feed-forward transformations. Module names depend on model architecture. Rank controls the intermediate dimension of the update; alpha contributes to its scale. Neither a larger rank nor a larger scale automatically means better task performance.

Learning rate governs the size of optimization changes, while the number of updates controls how long fitting continues. Sequence length determines how much context is retained. Microbatch size determines how many examples are processed together and affects memory use. Validation schedule determines when selection evidence is collected. The seed helps make data ordering and stochastic choices reproducible, although identical hardware and software conditions also matter.

Caveat or limitation

The lecture does not establish that query/value updates are optimal for alpha, risk or extraction. Choosing them in a run is a design choice that must be tested.

Transition

Next we will consider this point: Memory-saving methods solve resource problems.

Slide 28: Memory-saving methods solve resource problems

Main goal in plain English

Each saves something different.

Natural opening and explanation

QLoRA keeps a quantized frozen base and trains adapters. Gradient accumulation combines smaller batches before updating. Checkpointing recomputes activations to save their memory.

QLoRA is more specific than arbitrary low-precision training. Gradient checkpointing is different from saving model checkpoints. Do a fit-step-save-reload smoke test first.

Concrete example

Accumulating eight small batches can replace one larger update batch. It cannot make a single oversized record fit.

Detailed explanation for delivery

"Suppose the GPU cannot hold a batch of eight records at once. We may process smaller groups and accumulate gradients before updating. If even one long record does not fit, accumulation alone will not solve it. We must address its length or another source of memory demand."

QLoRA trains low-rank adapters while the pretrained base is stored in a quantized form. Quantization uses a more compact numerical representation for weights, reducing their storage requirement. QLoRA includes specific quantization and optimizer techniques; it is more precise than simply saying "any low-precision training."

Gradient checkpointing saves less intermediate activation state and recomputes some of it during backpropagation. It exchanges additional computation for lower activation memory. This use of "checkpointing" differs from saving model checkpoints for later selection.

Caveat or limitation

Resource feasibility and model quality are separate. A run that fits memory and completes quickly may still be poorly designed or produce a worse candidate.

Transition

Next we will consider this point: What one training step actually does.

Slide 29: What one training step actually does

Main goal in plain English

What one training step actually does

Natural opening and explanation

This is the conceptual sequence behind a trainer call. Production code also handles accumulation, scaling and scheduling. Validation measures outputs without performing these parameter updates.

Detailed explanation for delivery

"For one batch of request examples, the model predicts target words, calculates a penalty, calculates gradients, updates the permitted parameters and clears the gradients. A validation pass predicts and measures but does not perform that update. That distinction tells us whether data is being used to fit or merely assess."

"This is a conceptual training loop. The library handles more details, but these lines tell us what actually changes."

Optimizer.step() uses those gradients and the optimizer's state to update parameters included in its parameter groups. In a LoRA run those should be the intended trainable adapter parameters. Optimizer.zero_grad() clears accumulated gradients so the next ordinary update does not unintentionally include old gradients. With deliberate gradient accumulation, clearing occurs at the appropriate accumulation interval instead.

Caveat or limitation

The slide is explanatory pseudocode, not a copy-paste recipe for every environment. The prepared instructor notebook uses its existing implementation and pinned dependencies.

Transition

Next we will consider this point: Saving and reloading is part of the experiment.

Slide 30: Saving and reloading is part of the experiment

Main goal in plain English

A metric must identify a reproducible candidate.

Natural opening and explanation

Save the adapter, compatible base revision, tokenizer, template, configuration and evaluation outputs. Check that the saved candidate reloads before relying on it.

A smoke run verifies mechanics; it does not qualify model quality. During validation there must be no optimizer step.

Concrete example

Two adapters with the same file name but different base revisions are not necessarily interchangeable.

Detailed explanation for delivery

"For one batch of request examples, the model predicts target words, calculates a penalty, calculates gradients, updates the permitted parameters and clears the gradients. A validation pass predicts and measures but does not perform that update. That distinction tells us whether data is being used to fit or merely assess."

Optimizer.step() uses those gradients and the optimizer's state to update parameters included in its parameter groups. In a LoRA run those should be the intended trainable adapter parameters. Optimizer.zero_grad() clears accumulated gradients so the next ordinary update does not unintentionally include old gradients. With deliberate gradient accumulation, clearing occurs at the appropriate accumulation interval instead.

The final comment highlights a separate activity: evaluate validation, select, save and reload. Evaluation should not call an optimizer update. A saved model must be loaded with the compatible tokenizer, template, base revision and adapter settings.

Caveat or limitation

The slide is explanatory pseudocode, not a copy-paste recipe for every environment. The prepared instructor notebook uses its existing implementation and pinned dependencies.

Transition

Next we will consider this point: Choose the route that answers your question.

Slide 31: Choose the route that answers your question

Main goal in plain English

Choose the route that answers your question

Natural opening and explanation

Replay, workflow intervention and fresh optimization are different activities. Each is useful, but none should be described as the other. The instructor route defaults to recorded mode.

Detailed explanation for delivery

“Watching the recorded adapter comparison is like examining a completed laboratory experiment. Repairing a supplied tool argument runs a controlled workflow again. Starting the optional trainer performs new parameter updates. Each can be hands-on, but they support different claims about what happened.”

The fresh optimization route is an instructor option. It performs actual gradient updates on prepared hardware when explicitly enabled. A short demonstration verifies mechanics and artifacts; it does not replace the larger qualification experiment. The instructor notebook defaults to the recorded route.

Tell participants to use the route matching the question: “What did this adapter change?” uses recorded comparisons. “What would this verified repair affect?” uses workflow diagnosis. “Can this environment perform and save the update?” uses a smoke demonstration.

Caveat or limitation

Do not describe a slider, fixture or replay as live neural training. Conversely, a recorded experiment can be genuine evidence of what happened in that run even though it is not being rerun during class.

Transition

Next we will consider this point: Exercise A: inspect what was trained.

Slide 32: Exercise A: inspect what was trained

Main goal in plain English

Open the readable adaptation notebook.

Natural opening and explanation

Run sections 1–2 of `04_actual_adaptation_readable.ipynb`. Find the input, target, trainable modules and selected epoch.

Run setup cells in order. The default loads saved artifacts and makes no live model calls. Do not mistake the observed outcome for a prediction-time feature.

Concrete example

Be able to finish: "The model received [input] and was trained to output [target]."

Detailed explanation for delivery

"We will inspect a real saved run in a deliberate order: what was trained, what changed, and whether the change was correct."

Ask participants to open 04_actual_adaptation_readable.ipynb and run the setup cells in order. In sections 1 and 2, locate a training input, its target, the selected modules and the selected epoch. Ask them to state whether the target is visible in the prediction-time input or is a separate supervised label.

In section 3, change CASE_INDEX to another valid case and rerun that cell. Before revealing the observed outcome, ask them to make their own classification from the supplied information. Then proceed to section 4 to reveal the reference outcome and inspect the recorded checks. Compare the base and adapter on exactly that same case.

Caveat or limitation

The historical target contract and outputs must be interpreted as recorded. Do not silently rewrite labels or select only favourable cases. All sixteen evaluation cases remain in the aggregate.

Transition

Next we will consider this point: Exercise A: compare one matched case.

Slide 33: Exercise A: compare one matched case

Main goal in plain English

Change the case, then reveal the outcome.

Natural opening and explanation

In section 3 change `CASE_INDEX`. Predict before revealing the reference outcome. Run sections 4–6 to inspect checks, aggregate results and split dates.

Keep the same evidence for both models. Then inspect all sixteen cases rather than selecting a winning subset. Saved answers cannot update if you change their evidence.

Concrete example

State whether the adapter's changed answer corrected an error or introduced one.

Detailed explanation for delivery

"Choose a saved case where the base and adapter differ. If the reference label is positive, a change from negative to positive is a correction; a change from positive to neutral is a regression. The reference must come from the defined evaluation, not from whichever answer we prefer."

In section 3, change CASE_INDEX to another valid case and rerun that cell. Before revealing the observed outcome, ask them to make their own classification from the supplied information. Then proceed to section 4 to reveal the reference outcome and inspect the recorded checks. Compare the base and adapter on exactly that same case.

Sections 5 and 6 recompute the aggregate and inspect temporal separation. Ask each pair to write one sentence: "On case X, the adapter changed Y to Z, which corrected/introduced/did not resolve an error." Ensure they do not infer overall improvement from one case.

Caveat or limitation

The historical target contract and outputs must be interpreted as recorded. Do not silently rewrite labels or select only favourable cases. All sixteen evaluation cases remain in the aggregate.

Transition

Next we will consider this point: The recorded run: what these results refer to.

Slide 34: The recorded run: what these results refer to

Main goal in plain English

The recorded run: what these results refer to

Natural opening and explanation

One model and adapter cover multiple stocks. Balancing does not create independent windows. These figures identify an existing run; they are not runtime guarantees for a new job.

Detailed explanation for delivery

"These are the identifying facts of the recorded run. They tell us what the subsequent numbers refer to."

The dataset contains 527 distinct training windows. Balancing produced 777 records per epoch, so some sampling or replication is involved. More balanced records are not the same as more independent information. One pass through this training material is an epoch. The recorded run took 28.6 minutes on one T4 GPU. That duration belongs to this workload and environment, not to all LoRA jobs.

The chosen checkpoint is epoch two, selected using validation loss. One model and adapter covered multiple stocks; it was not a separate model for each stock. Explain that this financial classification experiment is different from the earlier illustrative current-status task. We are using it because its actual artifacts are available for inspection.

Caveat or limitation

The base model's advertised size is approximate. Quote the recorded count when discussing this run. Do not infer a whole-model trainable percentage from the single-matrix arithmetic example.

Transition

Next we will consider this point: A real labelled record, condensed for readability.

Slide 35: A real labelled record, condensed for readability

Main goal in plain English

A real labelled record, condensed for readability

Natural opening and explanation

This is condensed from the saved training examples record, not an invented pair. The full notebook record retains all fields. Future return supplies the supervised label after it matures; it is not part of the earlier prediction input.

Detailed explanation for delivery

The chosen checkpoint is epoch two, selected using validation loss. One model and adapter covered multiple stocks; it was not a separate model for each stock. Explain that this financial classification experiment is different from the earlier illustrative current-status task. We are using it because its actual artifacts are available for inspection.

The following is an explicitly condensed view of the first record in `training_examples` in the existing `recorded-adaptation.json` artifact. It is not a newly invented training pair. The notebook retains the complete record.

> Entity and cutoff: CVX; 2025-10-09 06:30 UTC.
> Input excerpt: 21 split-adjusted daily closing prices ending 2025-10-08; latest close USD 153.7400; prior 20-session price return −2.5667%. Other supplied fields include news, financial facts and SPY context.
> System label mapping: A = excess return below −1%; B = −1% through +1%; C = above +1%.
> Recorded target output: `C`.

"What is learned is the probability of the class token, and the configured end token, conditional on that evidence and instruction. The target is based on a subsequently realized five-session return relative to SPY. It is not a label saying the evidence is complete, that the news is relevant, or that a trade is authorized. The future return defines a supervised target only after it is observed; it must not enter the earlier prediction-time input."

The saved record also exposes a contract inconsistency: its nested question requests named signal labels and abstention, while the system instruction requests exactly A, B or C. Preserve and discuss that historical inconsistency rather than silently editing it. The supplied news concerns Sintana and Challenger, so its presence does not itself establish CVX relevance. This is a concrete reason to inspect training inputs rather than equating a timestamp check with adequate evidence. Training loss can improve while the model learns from a noisy or poorly aligned input-target pairing.

Caveat or limitation

The base model's advertised size is approximate. Quote the recorded count when discussing this run. Do not infer a whole-model trainable percentage from the single-matrix arithmetic example.

Transition

Next we will consider this point: The financial target is not a truth certificate.

Slide 36: The financial target is not a truth certificate

Main goal in plain English

The model learns a class-token mapping.

Natural opening and explanation

The target does not certify evidence quality, predict a profitable authorized trade or provide calibrated confidence. Inspect the exact historical prompt contract.

The first saved news item concerns other companies; presence and freshness do not prove issuer relevance. Preserve these limitations rather than silently rewriting the record.

Concrete example

The saved system requests A/B/C, while its nested question requests named signals and abstention. That mismatch is an audit finding.

Detailed explanation for delivery

The chosen checkpoint is epoch two, selected using validation loss. One model and adapter covered multiple stocks; it was not a separate model for each stock. Explain that this financial classification experiment is different from the earlier illustrative current-status task. We are using it because its actual artifacts are available for inspection.

The following is an explicitly condensed view of the first record in `training_examples` in the existing `recorded-adaptation.json` artifact. It is not a newly invented training pair. The notebook retains the complete record.

> Entity and cutoff: CVX; 2025-10-09 06:30 UTC.
> Input excerpt: 21 split-adjusted daily closing prices ending 2025-10-08; latest close USD 153.7400; prior 20-session price return −2.5667%. Other supplied fields include news, financial facts and SPY context.
> System label mapping: A = excess return below −1%; B = −1% through +1%; C = above +1%.
> Recorded target output: `C`.

"What is learned is the probability of the class token, and the configured end token, conditional on that evidence and instruction. The target is based on a subsequently realized five-session return relative to SPY. It is not a label saying the evidence is complete, that the news is relevant, or that a trade is authorized. The future return defines a supervised target only after it is observed; it must not enter the earlier prediction-time input."

The saved record also exposes a contract inconsistency: its nested question requests named signal labels and abstention, while the system instruction requests exactly A, B or C. Preserve and discuss that historical inconsistency rather than silently editing it. The supplied news concerns Sintana and Challenger, so its presence does not itself establish CVX relevance. This is a concrete reason to inspect training inputs rather than equating a timestamp check with adequate evidence. Training loss can improve while the model learns from a noisy or poorly aligned input-target pairing.

Caveat or limitation

The base model's advertised size is approximate. Quote the recorded count when discussing this run. Do not infer a whole-model trainable percentage from the single-matrix arithmetic example.

Transition

Next we will consider this point: Why epoch two was selected.

Slide 37: Why epoch two was selected

Main goal in plain English

Why epoch two was selected

Natural opening and explanation

Logged means depend on encountered batches and weighting; they are not a separately rescored frozen-base baseline. The saved logs retain three non-finite gradient diagnostics as a limitation. Evaluated candidate logits are finite.

Detailed explanation for delivery

Read the training row from left to right: 0.3783, 0.3563 and 0.3510. These are mean logged training losses for each epoch. They indicate the recorded fit trajectory during optimization. They are not an independently evaluated unadapted-model loss on an identical frozen dataset.

Then read validation: 0.4040, 0.3692 and 0.3770. The lowest validation loss is at epoch two. Under the declared validation-loss rule, that is the selected checkpoint even though epoch three has lower training loss. Ask participants to explain that distinction in plain English: "The model fit the training material more closely, but the separate selection material stopped improving."

A meaningful comparison needs compatible target definitions, masking, reduction and evaluation data. Logged training losses are affected by which batches and sampled records were encountered; oversampling affects their weighting. The earlier validation set selects the checkpoint, but repeated choice among many configurations can still overfit that selection set. In this artifact, the target is a class token plus end token, not a long narrative. Lower token loss should not be interpreted as better explanation quality or calibrated return uncertainty. The recorded logs retain three non-finite gradient-norm diagnostics as an audit limitation; the saved evaluated candidate logits were finite.

Caveat or limitation

These values do not establish which internal layer caused any improvement or failure. No new training was performed to generate this slide.

Transition

Next we will consider this point: Count corrections and regressions together.

Slide 38: Count corrections and regressions together

Main goal in plain English

Count corrections and regressions together

Natural opening and explanation

Six original correct minus six regressions plus three corrections leaves three correct. Inspect both corrected and broken cases. No significance or investment performance claim follows from these small counts.

Detailed explanation for delivery

Start with the first row: the base is correct on six of sixteen cases; the adapter on three. That is a decrease in ordinary accuracy. Balanced accuracy averages recall across the outcome classes, giving each class equal weight. It rises from 33.3 percent to 40.7 percent here, which can appear contradictory until we inspect the class distribution.

The paired rows make the practical change clearer: the adapter fixes three errors the base made, but breaks six cases the base got right. The arithmetic is six original correct minus six regressions plus three corrections equals three adapter correct. A useful review examines examples from both groups.

Do not claim statistical significance or market alpha from these counts. The saved class outputs do not include model-generated intermediate reasoning that can be reconstructed retrospectively.

Caveat or limitation

Do not claim statistical significance or market alpha from these counts. The saved class outputs do not include model-generated intermediate reasoning that can be reconstructed retrospectively.

Transition

Next we will consider this point: Why a different metric can appear to improve.

Slide 39: Why a different metric can appear to improve

Main goal in plain English

Balanced accuracy gives classes equal weight.

Natural opening and explanation

It rises from 33.3% to 40.7%, while ordinary accuracy falls. Here there are six negative outcomes, one neutral and nine positive. The single neutral case has large influence.

Accuracy weights cases equally; balanced accuracy averages class recall. Neither automatically weights financial consequences correctly. The sixteen cases are correlated, so they are not sixteen independent confirmations.

Concrete example

Correctly recognizing that one neutral outcome contributes one third of the class-averaged metric.

Detailed explanation for delivery

Start with the first row: the base is correct on six of sixteen cases; the adapter on three. That is a decrease in ordinary accuracy. Balanced accuracy averages recall across the outcome classes, giving each class equal weight. It rises from 33.3 percent to 40.7 percent here, which can appear contradictory until we inspect the class distribution.

There are six negative, one neutral and nine positive outcomes. Correctly identifying the single neutral case has a large effect on an equal-class-weighted average. That is why metric choice and class counts matter. This small dataset is not enough to make a broad superiority claim.

Ordinary accuracy weights each case equally. Balanced accuracy weights each outcome class equally by averaging class-specific recall: the fraction of true cases in a class correctly recognized. That is why one neutral case can strongly influence balanced accuracy here. Neither weighting is automatically the economically relevant one. Correlated cases also mean sixteen rows need not contain sixteen independent pieces of evidence. These metrics measure agreement with the defined outcome labels, not evidence quality, calibration or strategy profit. The class outputs are selected from A/B/C scores at the same prompt position; normalized scores over those choices are not automatically calibrated outcome probabilities.

Caveat or limitation

Do not claim statistical significance or market alpha from these counts. The saved class outputs do not include model-generated intermediate reasoning that can be reconstructed retrospectively.

Transition

Next we will consider this point: The final answer does not locate the failure.

Slide 40: The final answer does not locate the failure

Main goal in plain English

Inspect the handoffs.

Natural opening and explanation

Follow evidence → extraction → tool arguments → calculation → allowed action. Check the recorded input and output at each step.

An evidence graph records these dependencies. It is not a display of hidden neural reasoning.

Concrete example

The source and extraction say approved, but the application's request says waiting. That mismatch is observable.

Detailed explanation for delivery

Walk left to right. Retrieved evidence is the actual source content available to the model. Model extraction is the field or claim selected from that evidence. Tool arguments are the exact values sent to a function. Calculation is the function's output. Allowed action is what the execution controller permits under declared requirements and authority.

For each arrow, ask what relationship should hold. An extracted field should be supported by the source. A tool argument should match the verified field and units. A calculation should match the arguments and formula. An action should satisfy the relevant policy and permissions. These checks can be recorded as dependencies.

A dependency graph makes downstream impact visible: if the extracted field changes, a calculation and an action depending on it may require reassessment. It does not mean every downstream value necessarily changes, and it does not expose the model's private reasoning or prove the unique neural cause of an error.

Caveat or limitation

The controlled workflow exercise and the sixteen-case saved model evaluation are different experiments. Do not use the controlled graph to claim a measured internal explanation of the saved model's predictions.

Transition

Next we will consider this point: Use the graph to target a repair.

Slide 41: Use the graph to target a repair

Main goal in plain English

Recheck what depends on the changed value.

Natural opening and explanation

If an input is repaired, reassess its dependent calculations and claims. Keep independent permissions and other faults visible.

BeliefLens organizes artifacts and declared checks. The controlled graph exercise does not establish the internal cause of the saved model's class predictions.

Concrete example

A corrected extraction can repair a report while a separate wrong calculator argument still blocks the action.

Detailed explanation for delivery

For each arrow, ask what relationship should hold. An extracted field should be supported by the source. A tool argument should match the verified field and units. A calculation should match the arguments and formula. An action should satisfy the relevant policy and permissions. These checks can be recorded as dependencies.

A dependency graph makes downstream impact visible: if the extracted field changes, a calculation and an action depending on it may require reassessment. It does not mean every downstream value necessarily changes, and it does not expose the model's private reasoning or prove the unique neural cause of an error.

The controlled workflow exercise and the sixteen-case saved model evaluation are different experiments. Do not use the controlled graph to claim a measured internal explanation of the saved model's predictions.

Caveat or limitation

The controlled workflow exercise and the sixteen-case saved model evaluation are different experiments. Do not use the controlled graph to claim a measured internal explanation of the saved model's predictions.

Transition

Next we will consider this point: Exercise B: inspect the actual calculator request.

Slide 42: Exercise B: inspect the actual calculator request

Main goal in plain English

Compare source values with tool arguments.

Natural opening and explanation

Open `04_workflow_diagnosis.ipynb` or web Section 4. Select `arguments`. The source says consensus 1.00, but the request passes 2.00.

Read the source first and then the exact request. Ask which input should change and which downstream output you expect to change.

Concrete example

Actual earnings: 1.20. Correct expected earnings: 1.00. The input discrepancy appears before the calculator runs.

Detailed explanation for delivery

"We will repair a tool request, then check whether the expected downstream quantities change."

Open 04_workflow_diagnosis.ipynb or the web Section 4 exercise. Select the arguments case. Ask participants to inspect the source first, then the actual calculator request. The supplied source has reported earnings per share of 1.20 and consensus of 1.00. The faulty request uses consensus 2.00.

Before applying repair, ask which checks and outputs should change. Then apply the verified tool-request replacement and inspect the recomputed result. Next select two_faults, where extraction and arguments are both wrong. Repair extraction alone and ask why the full workflow remains blocked. A separate argument defect persists.

Caveat or limitation

The reference values and injected faults are supplied by the exercise. The tool is not independently discovering profitable trading rules. A permitted simulated action does not demonstrate an investment edge.

Transition

Next we will consider this point: Correct arithmetic can use the wrong input.

Slide 43: Correct arithmetic can use the wrong input

Main goal in plain English

Correct arithmetic can use the wrong input

Natural opening and explanation

Inputs must share company, period, units and basis; consensus must be nonzero. Near-zero denominators are unstable. The result is a deterministic statistic, not evidence of an investment edge.

Detailed explanation for delivery

Open 04_workflow_diagnosis.ipynb or the web Section 4 exercise. Select the arguments case. Ask participants to inspect the source first, then the actual calculator request. The supplied source has reported earnings per share of 1.20 and consensus of 1.00. The faulty request uses consensus 2.00.

Explain the calculation. Surprise equals actual minus consensus, divided by the absolute consensus. Correct inputs give $(1.20 - 1.00)/1.00 = 0.20$, or twenty percent. The wrong input gives $(1.20 - 2.00)/2.00 = -0.40$, or minus forty percent. The calculator is arithmetically correct for the arguments it received.

For $(actual - consensus)/|consensus|$, both inputs must refer to the same company, reporting period, units and earnings basis, and consensus must be nonzero. The absolute value keeps the denominator's sign from reversing the interpretation, but values near zero can make percentage surprise unstable. In this supplied fixture, the positive consensus values 1.00 and 2.00 make the arithmetic simple. The calculation is a deterministic transformation of assumed-valid inputs; it does not establish that a positive surprise predicts a profitable trade. The fixture's action threshold is a declared illustrative rule, not an estimated investment relationship.

Caveat or limitation

The reference values and injected faults are supplied by the exercise. The tool is not independently discovering profitable trading rules. A permitted simulated action does not demonstrate an investment edge.

Transition

Next we will consider this point: Exercise B: repair one fault, then inspect what remains.

Slide 44: Exercise B: repair one fault, then inspect what remains

Main goal in plain English

Predict before applying the repair.

Natural opening and explanation

Apply the tool-request repair. Then try `two_faults`: repair extraction alone and inspect the remaining argument defect.

These are supplied reference values and controlled faults. The graph recomputes dependencies; it does not magically repair every downstream issue.

Concrete example

A repaired figure can restore one report while the trading path remains withheld.

Detailed explanation for delivery

"We will repair a tool request, then check whether the expected downstream quantities change."

Before applying repair, ask which checks and outputs should change. Then apply the verified tool-request replacement and inspect the recomputed result. Next select two_faults, where extraction and arguments are both wrong. Repair extraction alone and ask why the full workflow remains blocked. A separate argument defect persists.

The reference values and injected faults are supplied by the exercise. The tool is not independently discovering profitable trading rules. A permitted simulated action does not demonstrate an investment edge.

Caveat or limitation

The reference values and injected faults are supplied by the exercise. The tool is not independently discovering profitable trading rules. A permitted simulated action does not demonstrate an investment edge.

Transition

Next we will consider this point: Question: which intervention addresses this defect?.

Slide 45: Question: which intervention addresses this defect?

Main goal in plain English

Question: which intervention addresses this defect?

Natural opening and explanation

Ask for a choice before showing the answer. The evidence is the mismatch between correct extraction and the request that the application constructed.

Detailed explanation for delivery

“The extraction is correct, but the application changes a calculator argument. What should we repair first?”

Ask participants to vote before showing the next slide. Then ask what observation supports their answer. The evidence should be the correct extracted value followed by a different tool argument. That pair of records identifies the failed observable contract.

If someone argues that a model could compensate for the defect, acknowledge that compensation is possible in some systems but would be fragile and would leave an incorrect interface in place. The desired engineering response is to correct the contract and test it.

Caveat or limitation

Do not treat a later correct final action as proof that the intermediate defect was repaired. The boundary must be checked directly.

Transition

Next we will consider this point: Answer: B, repair the argument handoff.

Slide 46: Answer: B, repair the argument handoff

Main goal in plain English

Verify the input-to-source relationship.

Natural opening and explanation

Correct the application request, then recompute affected outputs. Retain unrelated faults and independent permission checks.

Training is a candidate for persistent interpretation errors with adequate context. A changed final answer alone does not prove the application defect was fixed.

Concrete example

An approved request may now be recorded correctly, while the operator still lacks authority to release it.

Detailed explanation for delivery

"After fixing the form submission, the stored status may become approved. That does not automatically give the operator permission to release the request. Correct content and authority are separate requirements, so we recheck affected outputs while retaining independent permission checks."

Explain binding in familiar terms: the value sent to the tool should be the value justified by the cited field, with the correct units, period and interpretation. Once that input is corrected, recompute the calculations and claims that depend on it. A graph helps identify which outputs need reassessment.

Keep unrelated faults visible. If a second extraction error or a missing permission remains, a tool-input repair should not silently mark the whole workflow as valid. Independent permissions must still pass even when the proposed action is now supported.

Caveat or limitation

Successful repair in a controlled fixture supports the implementation's declared dependency behaviour. It does not establish that every production failure has the same cause or that the graph discovers unknown requirements automatically.

Transition

Next we will consider this point: Decide using the full experimental evidence.

Slide 47: Decide using the full experimental evidence

Main goal in plain English

Accept, revise or reject for a defined scope.

Natural opening and explanation

Review task performance, uncertainty, regression cases, evidence checks and execution permissions. A falling loss curve is only one part of that record.

There is no universal threshold. Consequences and evidence quantity determine what is sufficient. The small recorded run does not support a general improvement claim.

Concrete example

A model that improves ordinary cases but invents unsupported approvals may need rejection or a narrower permitted use.

Detailed explanation for delivery

Task metric and uncertainty ask whether the selected candidate improves the intended behaviour and how much uncertainty remains because of sample size or variability. Regression slices ask where previously supported behaviour deteriorates, including missing evidence and unfamiliar inputs. Workflow validity asks whether evidence support, tool inputs and permissions still satisfy their requirements.

Explain the three possible outcomes. Accept means the evidence meets predeclared requirements for a defined scope. Revise means a concrete unresolved problem motivates another experiment. Reject means this candidate should not be used for the intended purpose. For the small recorded comparison, do not approve a general improvement claim merely because behaviour changed or validation loss fell.

There is no universal acceptance threshold. Consequences, tolerance for errors and the intended population determine the required evidence.

Caveat or limitation

Artifact completeness does not repair weak experimental design. A perfectly reproducible biased evaluation remains biased. More detailed testing is justified when it resolves a specific decision uncertainty.

Transition

Next we will consider this point: Make the decision reproducible and accountable.

Slide 48: Make the decision reproducible and accountable

Main goal in plain English

Keep the artifacts and name the owner.

Natural opening and explanation

Record data and split versions, model and adapter revisions, configuration, outputs and limitations. Specify permitted scope, monitoring and rollback.

Reproducibility cannot cure a biased comparison; it makes the evidence inspectable. A new experiment should resolve a specific remaining uncertainty.

Concrete example

Another reviewer should be able to identify the selected candidate and explain why it was accepted or rejected.

Detailed explanation for delivery

Reproducible artifacts should identify the data version, splits, base revision, adapter, tokenizer, template, configuration, evaluation code and outputs. Another reviewer should be able to understand and replay the comparison. Release decision assigns an accountable owner and specifies allowed scope, monitoring and a rollback route.

Explain the three possible outcomes. Accept means the evidence meets predeclared requirements for a defined scope. Revise means a concrete unresolved problem motivates another experiment. Reject means this candidate should not be used for the intended purpose. For the small recorded comparison, do not approve a general improvement claim merely because behaviour changed or validation loss fell.

Artifact completeness does not repair weak experimental design. A perfectly reproducible biased evaluation remains biased. More detailed testing is justified when it resolves a specific decision uncertainty.

Caveat or limitation

Artifact completeness does not repair weak experimental design. A perfectly reproducible biased evaluation remains biased. More detailed testing is justified when it resolves a specific decision uncertainty.

Transition

Next we will consider this point: Explain your conclusion in three sentences.

Slide 49: Explain your conclusion in three sentences

Main goal in plain English

Explain your conclusion in three sentences

Natural opening and explanation

An adequate response can conclude that training is unnecessary. This closes the five objectives: specify, explain the update, protect evaluation, compare behaviour and choose the repair.

Detailed explanation for delivery

For specifying the target, explain the input, reviewed response and supervised-token mask. For explaining the update, identify frozen base weights and trainable A and B matrices, including why the adapter still needs the base. For protecting evaluation, distinguish records that fit parameters, records that select a checkpoint and records that assess the frozen choice.

For comparing behaviour, describe corrections and regressions on matched cases. A changed label is not automatically an improvement. For choosing the repair, name the failed observable handoff and the intervention tested. The argument exercise should provide a concrete example.

Give the exit ticket: "State one candidate change and the evidence you would require to accept it." A good response might specify training on reviewed current-status examples, assessing unseen records with exceptions, counting regressions and verifying supported extraction at the tool handoff. Another good response may conclude that no training is needed because the defect is in application code.

Caveat or limitation

Do not require everyone to master all implementation details during one hour. The minimum is an accurate explanation of what changed and what evidence supports using it.

Transition

Next we will consider this point: Predictions over time and their financial consequences.